



Lista de Revisão 3

Orientações para resolução da lista

- As questões abordam Recursividade, Tratamento de Exceções, Leitura e Escrita de Arquivos e Orientação a Objetos.
- Recomenda-se o planejamento da lógica, casos base e modelagem de classes antes da implementação do código.
- Para questões de tratamento de exceções, foque na prevenção de quebra abrupta de execução.
- Utilize `try/except/finally` adequadamente.

Parte 1: Questões Abertas

1. Um laboratório mantém o arquivo texto `leituras.txt`, contendo medições realizadas por diferentes sensores. Cada linha válida do arquivo possui o seguinte formato:

```
codigo_sensor;data;temperatura
```

Por exemplo:

```
S01;10/06/2026;24.5  
S02;10/06/2026;27.3  
S01;11/06/2026;26.0  
S03;11/06/2026;erro  
S02;12/06/2026;28.1
```

Desenvolva um programa que leia o arquivo e produza um relatório contendo, para cada sensor:

- a quantidade de medições válidas;
- a menor temperatura registrada;
- a maior temperatura registrada;
- a temperatura média.

Linhas que não possuam exatamente três campos ou cuja temperatura não possa ser convertida para um número real devem ser consideradas inválidas.

Ao final, o programa deve:

- exibir a quantidade total de linhas inválidas;
- identificar o sensor que apresentou a maior temperatura entre todas as medições;
- gravar os resultados no arquivo `relatorio_sensores.txt`.

O relatório deve apresentar um sensor por linha. Não considere que os códigos dos sensores sejam previamente conhecidos.

2. Dois arquivos, chamados `nomes1.txt` e `nomes2.txt`, armazenam nomes de pessoas em ordem alfabética, com um nome em cada linha. Os arquivos podem possuir quantidades diferentes de nomes.

Desenvolva um programa que crie o arquivo `nomes_mesclados.txt`, contendo todos os nomes dos dois arquivos também em ordem alfabética.

A mesclagem deve ser realizada comparando a linha atual de cada arquivo, de maneira semelhante à etapa de intercalação de duas sequências ordenadas.

Para esta questão:

- não utilize a função `sort()` ou o método `sort()`;
- não reúna inicialmente todos os nomes em uma única lista para depois ordená-los;
- nomes repetidos nos dois arquivos devem aparecer apenas uma vez no arquivo resultante;
- ao final de um dos arquivos, todas as linhas ainda não processadas do outro arquivo devem ser copiadas.

Além de gerar o novo arquivo, o programa deve informar:

- quantos nomes foram lidos do primeiro arquivo;
- quantos nomes foram lidos do segundo arquivo;
- quantos nomes distintos foram gravados no arquivo resultante.

3. A secretaria de uma universidade armazena as notas dos alunos no arquivo `notas.txt`. Cada linha apresenta os dados de um aluno no seguinte formato:

```
matricula;nome;nota1;nota2;nota3
```

Desenvolva um programa que leia o arquivo e calcule a média de cada aluno. Considere a seguinte classificação:

- média maior ou igual a 70: Aprovado;

- média maior ou igual a 40 e menor que 70: Exame Especial;
- média menor que 40: Reprovado.

O programa deve gerar o arquivo `resultado.txt`, contendo, em cada linha:

```
matricula;nome;media;situacao
```

Além disso, o programa deve exibir:

- a média geral da turma;
- o nome e a média do aluno com maior desempenho;
- a quantidade de alunos em cada situação;
- a porcentagem de alunos aprovados;
- a quantidade de registros inválidos.

Um registro deve ser considerado inválido quando não possuir os cinco campos esperados, quando alguma nota não for numérica ou quando alguma nota estiver fora do intervalo de 0 a 100. Registros inválidos não devem participar dos cálculos.

4. Crie uma função recursiva chamada *soma_ate(n)* que receba um número inteiro positivo n e retorne a soma de todos os números de 1 até n .

Por exemplo:

```
soma_ate(5) -> 15
soma_ate(3) -> 6
soma_ate(0) -> 0
```

5. Crie uma função recursiva chamada *potencia(base, expoente)* que receba dois números inteiros positivos e retorne o valor de base elevado a expoente.

Por exemplo:

```
potencia(2,0) -> 1
potencia(4,1) -> 4
potencia(2,10) -> 1024
```

6. Crie uma função recursiva chamada *contar_digitos(n)* que receba um número inteiro positivo n e retorne a quantidade de dígitos desse número.

Por exemplo:

```
soma_digitos(4725) -> 4
soma_digitos(90) -> 2
soma_digitos(7) -> 1
```

7. Crie uma função recursiva chamada *existe(lista, valor, i)* que verifique se valor aparece na lista a partir da posição *i*. Se o valor existir na lista, a função retorna *True*, e caso contrário, retorna *False*.

Por exemplo:

```
existe([1,2,4,6,0],2,0) -> True
existe([1,2,4,6,0],2,9) -> False
```

8. Crie uma função recursiva chamada *soma_digitos(numero)* que receba um número inteiro não negativo e retorne a soma de seus algarismos.

Por exemplo:

```
soma_digitos(4725) -> 18
soma_digitos(90)   -> 9
soma_digitos(7)    -> 7
```

A função deve obter o último algarismo utilizando o resto da divisão por 10 e reduzir o número por meio da divisão inteira por 10.

Não utilize:

- estruturas de repetição dentro da função;
- conversão do número para string;
- listas ou variáveis globais.

No programa principal, leia vários números inteiros não negativos. A leitura deve ser encerrada quando o usuário informar um número negativo. Para cada número válido, exiba a soma de seus algarismos.

9. O algoritmo de Euclides permite calcular o máximo divisor comum de dois números inteiros utilizando sucessivas operações de resto.

Crie uma função recursiva chamada *calcular_mdc(a, b)* que utilize as seguintes regras:

- quando *b* for igual a zero, o resultado será *a*;
- caso contrário, o resultado será obtido por meio de uma nova chamada com os valores *b* e *a % b*.

Por exemplo:

```
calcular_mdc(48, 18) -> 6
calcular_mdc(100, 25) -> 25
calcular_mdc(17, 5) -> 1
```

Depois, crie um programa principal que leia pares de números positivos até que o primeiro número informado seja zero. Para cada par, exiba:

- o máximo divisor comum;
- se os números são coprimos;
- o mínimo múltiplo comum, calculado a partir do MDC.

Dois números são considerados coprimos quando seu máximo divisor comum é igual a 1.

10. Crie uma função recursiva chamada

```
maior_elemento(valores, indice)
```

que receba uma lista de números inteiros e um índice e retorne o maior elemento existente entre a posição indicada e o final da lista.

A chamada inicial deverá ser realizada da seguinte forma:

```
maior_elemento(valores, 0)
```

A função deve comparar o elemento da posição atual com o maior elemento encontrado nas posições seguintes.

Não utilize:

- estruturas de repetição dentro da função;
- as funções `max()` ou `sorted()`;
- cópias ou fatiamentos da lista;
- variáveis globais.

No programa principal, leia uma lista com N números inteiros e exiba:

- o maior elemento;
- a primeira posição em que o maior elemento aparece;
- a quantidade de vezes que ele aparece na lista.

A contagem das ocorrências do maior elemento também deve ser realizada por uma função recursiva.

11. Crie uma função recursiva chamada

```
contar_ocorrencias(valores, procurado, indice)
```

que receba uma lista de números inteiros, um valor procurado e a posição a partir da qual a busca deve ser realizada.

A função deve retornar quantas vezes o valor procurado aparece na lista.

Por exemplo:

```
valores = [4, 7, 4, 2, 4, 9]

contar_ocorrencias(valores, 4, 0) -> 3
contar_ocorrencias(valores, 8, 0) -> 0
```

Além da função de contagem, crie uma segunda função recursiva chamada

```
primeira_posicao(valores, procurado, indice)
```

que retorne o índice da primeira ocorrência do valor procurado ou -1 , caso ele não esteja presente.

No programa principal, leia a lista e um valor a ser pesquisado. Exiba a quantidade de ocorrências e a primeira posição encontrada.

Não utilize os métodos `count()` e `index()`.

- 12.** Crie uma função recursiva chamada `multiplicar(a, b)` que calcule o produto de dois números inteiros sem utilizar o operador de multiplicação.

A multiplicação deve ser realizada por meio de somas ou subtrações sucessivas. A função deve tratar corretamente os seguintes casos:

- os dois números são positivos;
- apenas um dos números é negativo;
- os dois números são negativos;
- um dos números é igual a zero.

Por exemplo:

```
multiplicar(5, 4)    -> 20
multiplicar(5, -4)   -> -20
multiplicar(-5, 4)   -> -20
multiplicar(-5, -4)  -> 20
multiplicar(8, 0)    -> 0
```

Não utilize:

- o operador `*`;
- estruturas de repetição dentro da função;
- conversões para string;
- variáveis globais.

No programa principal, leia diferentes pares de valores e compare o resultado produzido pela função recursiva com o resultado esperado.

- 13.** Crie uma função recursiva chamada

```
eh_primo(numero, divisor)
```

que determine se um número inteiro é primo.

A chamada inicial deve utilizar o divisor 2:

```
eh_primo(numero, 2)
```

A função deve utilizar as seguintes regras:

- números menores que 2 não são primos;
- se algum divisor produzir resto igual a zero, o número não é primo;
- se o quadrado do divisor atual for maior que o número, nenhum outro divisor precisa ser testado e o número é primo;
- caso contrário, a função deve testar o próximo divisor.

Não utilize estruturas de repetição dentro da função.

No programa principal, leia dois números inteiros positivos, representando o início e o final de um intervalo. Utilize a função recursiva para verificar cada número do intervalo e exiba todos os números primos encontrados, além da quantidade total de primos.

14. Uma aplicação precisa compactar pequenos textos substituindo sequências de caracteres iguais e consecutivos pelo caractere seguido da quantidade de repetições.

Por exemplo:

```
"aaabbcccccdaa" -> "a3b2c4d1a2"  
"xxxx"          -> "x4"  
"abc"           -> "a1b1c1"
```

Crie uma função recursiva chamada

```
compactar_texto(texto, indice, caractere_atual, quantidade)
```

que percorra o texto e retorne sua forma compactada.

A função deve:

- comparar o caractere da posição atual com o caractere que está sendo contabilizado;
- aumentar a quantidade quando os caracteres forem iguais;
- acrescentar o caractere e sua quantidade ao resultado quando uma nova sequência for iniciada;
- incluir a última sequência de caracteres ao chegar ao final do texto;
- tratar corretamente uma string vazia.

Não utilize:

- estruturas de repetição dentro da função recursiva;
- bibliotecas externas de compactação;
- variáveis globais.

Crie também uma função recursiva chamada `descompactar_texto(texto, indice)` que realize a operação inversa.

Por exemplo:

`"a3b2c4d1a2" -> "aaabbccccdaa"`

Considere que cada quantidade será um número inteiro entre 1 e 9.

No programa principal:

- leia textos até que seja informada a palavra "fim";
- exiba o texto compactado;
- descompacte o resultado;
- verifique se o texto descompactado é igual ao texto original;
- informe se a compactação reduziu, aumentou ou manteve o tamanho do texto.

15. Um robô foi colocado em um labirinto representado por uma matriz. Cada posição da matriz contém um dos seguintes valores:

- 0: posição livre;
- 1: parede;
- 2: posição inicial do robô;
- 3: posição de saída.

O robô pode se movimentar uma posição por vez para cima, para baixo, para a esquerda ou para a direita.

Crie uma função recursiva chamada

`encontrar_saida(labirinto, linha, coluna)`

que utilize a técnica de tentativa e retrocesso (*backtracking*) para encontrar um caminho entre a posição inicial e a saída.

A função deve:

- retornar `False` ao acessar uma posição fora dos limites da matriz;
- retornar `False` ao encontrar uma parede;
- retornar `False` ao encontrar uma posição já visitada;

- retornar `True` ao alcançar a saída;
- marcar temporariamente as posições visitadas;
- tentar os quatro movimentos possíveis;
- desfazer a marcação quando uma posição não fizer parte do caminho que leva à saída.

Quando um caminho for encontrado, as posições que fazem parte dele devem ser marcadas com o valor 4.

No programa principal:

- leia as dimensões e os valores do labirinto;
- localize a posição inicial do robô;
- chame a função recursiva;
- informe se existe um caminho até a saída;
- exiba a matriz com o caminho encontrado;
- informe a quantidade de posições percorridas no caminho.

Não utilize estruturas de repetição dentro da função recursiva. As estruturas de repetição podem ser utilizadas no programa principal para ler e exibir a matriz.

- 16.** Crie uma função recursiva chamada `converter_binario(n)` que receba um número inteiro não negativo e retorne uma string contendo sua representação em base binária.

Por exemplo:

```
converter_binario(13) -> "1101"
converter_binario(8)  -> "1000"
converter_binario(0)  -> "0"
```

A função deve determinar cada dígito binário utilizando divisões sucessivas por 2. Não utilize:

- as funções `bin()` ou `format()`;
- estruturas de repetição dentro da função;
- listas para armazenar os restos;
- variáveis globais.

Depois, crie um programa principal que leia vários números inteiros. A leitura deve ser encerrada quando o usuário informar um número negativo. Para cada número válido, o programa deve exibir:

- sua representação binária;
- a quantidade de algarismos binários produzidos;

- a quantidade de dígitos iguais a 1.

A contagem de dígitos iguais a 1 também deve ser realizada por uma função recursiva.

17. Desenvolva uma função recursiva chamada

```
eh_palindromo(texto, inicio, fim)
```

que determine se um texto é um palíndromo. A função deve comparar os caracteres localizados nas posições `inicio` e `fim` e continuar a verificação em direção ao centro do texto.

Durante a análise, a função deve:

- desconsiderar diferenças entre letras maiúsculas e minúsculas;
- ignorar espaços;
- ignorar os caracteres de pontuação `.`, `,`, `:`, `;`, `!` e `?`;
- retornar `True` quando o texto for um palíndromo e `False` caso contrário.

Exemplos de textos que devem ser reconhecidos como palíndromos:

```
"Socorram-me, subi no onibus em Marrocos"
"Apos a sopa"
"Anotaram a data da maratona"
```

Não crie uma cópia invertida do texto e não utilize estruturas de repetição dentro da função recursiva.

No programa principal, leia frases até que seja informada a palavra `"fim"`. Ao final, exiba a quantidade de frases palíndromas e não palíndromas analisadas.

18. Implemente uma classe chamada `ContaBancaria`. Cada objeto deve possuir os seguintes atributos privados:

- número da conta;
- nome do titular;
- saldo;
- limite de crédito;
- histórico de operações.

O construtor deve receber o número da conta, o nome do titular e o limite de crédito. Toda conta deve ser criada com saldo inicial igual a zero e histórico vazio.

Implemente os seguintes métodos:

- `depositar(valor)`: adiciona um valor positivo ao saldo;

- `sacar(valor)`: realiza o saque apenas quando o valor for positivo e não ultrapassar a soma do saldo com o limite;
- `transferir(destino, valor)`: retira o valor da conta atual e o deposita em outro objeto da classe `ContaBancaria`;
- `consultar_saldo()`: retorna o saldo atual;
- `consultar_saldo_disponivel()`: retorna a soma do saldo com o limite;
- `exibir_extrato()`: apresenta todas as operações realizadas na conta.

Cada operação válida deve ser registrada no histórico, indicando seu tipo e valor. Operações inválidas não devem alterar o saldo nem o histórico.

No programa principal:

- crie pelo menos três contas;
- realize depósitos, saques e transferências entre elas;
- tente executar pelo menos uma operação inválida;
- exiba o extrato e o saldo final de cada conta;
- identifique a conta com maior saldo disponível.

Os atributos não devem ser acessados ou modificados diretamente fora da classe.

19. Uma loja deseja representar seus pedidos utilizando composição de classes.

Crie inicialmente uma classe chamada `Produto`, contendo os atributos privados:

- código;
- descrição;
- preço unitário;
- quantidade disponível em estoque.

A classe deve possuir métodos para consultar seus dados, acrescentar unidades ao estoque e retirar unidades do estoque. Uma retirada somente pode ocorrer quando houver quantidade suficiente.

Em seguida, crie uma classe chamada `ItemPedido`, contendo:

- um objeto da classe `Produto`;
- a quantidade solicitada.

A classe deve possuir um método que calcule o subtotal do item.

Por fim, crie uma classe chamada `Pedido`, contendo:

- número do pedido;
- nome do cliente;
- uma lista de objetos da classe `ItemPedido`;

- situação do pedido, inicialmente definida como "Aberto".

Implemente na classe `Pedido` os métodos:

- `adicionar_item(produto, quantidade);`
- `remover_item(codigo_produto);`
- `calcular_total();`
- `finalizar();`
- `exibir_resumo();`

Ao finalizar um pedido, o estoque de cada produto deve ser reduzido. O pedido somente poderá ser finalizado se houver estoque suficiente para todos os itens. Caso um único item não possa ser atendido, nenhum estoque deverá ser alterado.

Depois de finalizado, o pedido não poderá receber nem remover itens.

No programa principal, crie diferentes produtos e pelo menos dois pedidos, incluindo uma tentativa de finalização sem estoque suficiente.

- 20.** Uma empresa possui diferentes categorias de funcionários. Implemente uma classe-base chamada `Funcionario`, contendo os atributos privados ou protegidos:

- matrícula;
- nome;
- salário-base.

A classe deve possuir:

- um construtor;
- métodos de acesso aos atributos;
- um método `calcular_salario()` que retorne o salário-base;
- um método `exibir_dados()`.

Crie as seguintes classes derivadas:

- `Gerente`, que possui uma gratificação fixa adicionada ao salário-base;
- `Vendedor`, que possui um valor total de vendas e recebe uma comissão percentual sobre esse valor;
- `Desenvolvedor`, que possui uma quantidade de horas extras e recebe um valor adicional por hora.

Cada classe derivada deve sobrescrever o método `calcular_salario()` de acordo com sua regra.

No programa principal:

- crie uma única lista contendo objetos das diferentes classes;

- percorra a lista e exiba os dados e o salário calculado de cada funcionário;
- calcule o total da folha de pagamento;
- identifique o funcionário com maior salário;
- calcule a média salarial da empresa;
- informe quantos funcionários recebem acima da média.

O cálculo realizado no percurso da lista deve utilizar o mesmo método `calcular_salario()`, independentemente da classe concreta do objeto.

- 21.** Uma empresa deseja desenvolver um sistema de controle de estoque que integre manipulação de arquivos, recursividade e programação orientada a objetos.

O arquivo `produtos.txt` armazena os produtos cadastrados, com uma linha no formato:

```
codigo;descricao;preco;quantidade
```

Crie uma classe chamada `Produto`, contendo atributos privados para os quatro campos do arquivo. A classe deve possuir:

- um construtor;
- métodos de acesso;
- um método para acrescentar unidades;
- um método para retirar unidades;
- um método para calcular o valor total do produto em estoque.

Crie também uma classe chamada `Estoque`, que mantenha uma lista de objetos da classe `Produto`.

A classe `Estoque` deve oferecer os seguintes métodos:

- cadastrar um novo produto;
- realizar a entrada de unidades;
- realizar a saída de unidades;
- calcular o valor financeiro total do estoque;
- identificar o produto com maior valor total armazenado;
- listar os produtos com quantidade abaixo de um limite informado.

A busca de um produto pelo código deve ser realizada obrigatoriamente por uma função recursiva com a seguinte assinatura:

```
localizar_produto(produtos, codigo, indice)
```

A função deve retornar o objeto encontrado ou `None` quando o código não existir. Não utilize estruturas de repetição dentro dessa função.

O programa principal deve:

- ler o arquivo `produtos.txt`;
- criar um objeto `Produto` para cada linha válida;
- armazenar os objetos em um objeto da classe `Estoque`;
- permitir a realização de entradas e saídas de produtos;
- impedir quantidades negativas e saídas superiores ao estoque disponível;
- apresentar o valor financeiro total do estoque;
- mostrar os produtos abaixo do estoque mínimo informado pelo usuário;
- gravar a situação final no arquivo `estoque_atualizado.txt`.

Cada linha do arquivo final deve manter o mesmo formato do arquivo original. Linhas inválidas do arquivo de entrada devem ser ignoradas e contabilizadas.

Ao final, o programa deve exibir:

- a quantidade de produtos carregados;
- a quantidade de registros inválidos;
- o produto de maior valor total em estoque;
- o valor financeiro total do estoque;
- a confirmação de que o arquivo atualizado foi gerado.

- 22.** Crie uma classe chamada `Livro` para representar um livro que está sendo lido por uma pessoa.

A classe deve possuir os seguintes atributos:

- título;
- autor;
- quantidade total de páginas;
- página atual da leitura.

O construtor deve receber o título, o autor e a quantidade total de páginas. Todo objeto deve ser criado com a página atual igual a zero.

Implemente os seguintes métodos:

- `avancar_paginas(quantidade)`: avança a leitura, sem permitir que a página atual ultrapasse o total de páginas;
- `voltar_paginas(quantidade)`: retorna páginas, sem permitir que a página atual fique negativa;
- `calcular_progresso()`: retorna a porcentagem do livro que já foi lida;

- `finalizado()`: retorna `True` quando todas as páginas tiverem sido lidas;
- `exibir_dados()`: apresenta os dados do livro e o progresso da leitura.

No programa principal, crie três objetos da classe `Livro`, realize diferentes operações de leitura e exiba os dados de cada objeto.

- 23.** Crie uma classe chamada `Produto` para representar um produto armazenado em uma loja.

A classe deve possuir os atributos:

- código;
- nome;
- preço unitário;
- quantidade em estoque.

Implemente um construtor e os seguintes métodos:

- `adicionar_estoque(quantidade)`;
- `retirar_estoque(quantidade)`;
- `alterar_preco(novo_preco)`;
- `calcular_valor_estoque()`;
- `exibir_produto()`.

A classe deve impedir:

- preços menores ou iguais a zero;
- quantidades negativas;
- retiradas superiores à quantidade disponível.

No programa principal, crie uma lista com cinco objetos da classe `Produto`. Depois, exiba:

- os dados de todos os produtos;
- o produto com maior quantidade em estoque;
- o produto com maior valor financeiro total em estoque;
- o valor total de todos os produtos armazenados.

- 24.** Crie uma classe chamada `Cronometro`, responsável por representar um intervalo de tempo por meio dos atributos:

- horas;
- minutos;
- segundos.

O construtor deve criar o cronômetro inicialmente com 0:0:0.

Implemente os métodos:

- `avancar_segundo()`: acrescenta um segundo ao horário;
- `retroceder_segundo()`: reduz um segundo, sem permitir valores negativos;
- `reiniciar()`: retorna o cronômetro para zero;
- `converter_para_segundos()`: retorna o tempo total em segundos;
- `exibir_tempo()`: apresenta o tempo no formato hh:mm:ss.

O método `avancar_segundo()` deve atualizar corretamente os minutos e as horas. Por exemplo, depois de 00:59:59, o tempo deve passar para 01:00:00.

No programa principal, crie dois cronômetros e realize operações diferentes em cada um, demonstrando que os objetos mantêm estados independentes.

25. Crie uma classe chamada `Aluno`, contendo os atributos:

- matrícula;
- nome;
- nota da primeira avaliação;
- nota da segunda avaliação;
- nota da terceira avaliação.

As notas devem estar no intervalo de 0 a 100.

Implemente os métodos:

- `alterar_nota(numero_avaliacao, nova_nota)`;
- `calcular_media()`;
- `obter_situacao()`;
- `exibir_boletim()`.

O método `obter_situacao()` deve retornar:

- "Aprovado", para média maior ou igual a 70;
- "Exame Especial", para média maior ou igual a 40 e menor que 70;
- "Reprovado", para média menor que 40.

No programa principal, crie uma lista de objetos da classe `Aluno` e exiba:

- o boletim de cada aluno;
- o aluno com maior média;
- a média geral da turma;

- a quantidade de alunos em cada situação.

26. Crie uma classe chamada `Elevador`. Antes de escrever o código, identifique quais informações devem representar o estado de um elevador.

Considere que o elevador precisa controlar:

- o andar atual;
- a quantidade total de andares do prédio;
- a capacidade máxima de pessoas;
- a quantidade atual de pessoas;
- se a porta está aberta ou fechada.

O elevador deve ser criado no térreo, vazio e com a porta fechada.

Defina e implemente métodos para permitir que o elevador:

- abra e feche a porta;
- receba a entrada de uma pessoa;
- permita a saída de uma pessoa;
- suba um andar;
- desça um andar;
- exiba seu estado atual.

As seguintes regras devem ser respeitadas:

- pessoas somente podem entrar ou sair com a porta aberta;
- o elevador não pode ultrapassar sua capacidade;
- o elevador não pode se movimentar com a porta aberta;
- o elevador não pode subir além do último andar;
- o elevador não pode descer abaixo do térreo.

No programa principal, crie um objeto da classe e simule uma sequência de operações válidas e inválidas.

27. Leia a descrição a seguir:

Uma clínica veterinária atende animais pertencentes a diferentes clientes. Para cada cliente, são registrados nome, CPF e telefone. Cada animal possui nome, espécie, raça, data de nascimento e peso. Um cliente pode possuir vários animais, mas cada animal pertence a apenas um cliente.

Durante uma consulta, o veterinário registra a data, o motivo do atendimento, o diagnóstico e o valor cobrado. Um animal pode passar por várias consultas. O sistema deve permitir cadastrar clientes e animais, alterar o peso de um animal, registrar uma consulta, consultar o histórico de atendimentos de um animal e calcular o valor total gasto pelo cliente.

Antes de implementar, faça a modelagem do sistema:

- a) Identifique as classes necessárias.
- b) Liste os atributos de cada classe.
- c) Liste os métodos que devem pertencer a cada classe.
- d) Indique os relacionamentos existentes entre as classes.
- e) Informe quais objetos devem armazenar listas de outros objetos.

Em seguida, implemente as classes propostas e crie um programa principal que:

- cadastre pelo menos dois clientes;
- cadastre animais para esses clientes;
- registre diferentes consultas;
- exiba o histórico de um animal;
- calcule o total gasto por cada cliente.

Evite criar uma única classe responsável por todas as operações. Cada comportamento deve ser colocado na classe que possui os dados necessários para realizá-lo.

28. Leia a descrição de um sistema de empréstimos de biblioteca:

A biblioteca mantém um catálogo de livros. Cada livro possui ISBN, título, autor e quantidade de exemplares disponíveis. Os usuários da biblioteca possuem matrícula, nome e uma lista de empréstimos.

Quando um usuário solicita um livro, o sistema deve verificar se existe um exemplar disponível. Caso exista, um empréstimo é criado contendo o livro, o usuário, a data do empréstimo, a data prevista para devolução e a situação. Quando o livro é devolvido, a situação do empréstimo deve ser alterada e a quantidade disponível deve ser atualizada.

Um usuário não pode manter mais de três empréstimos ativos. O sistema também deve permitir consultar os empréstimos ativos de um usuário e verificar quais livros estão indisponíveis.

Com base no texto:

- a) Identifique pelo menos três classes necessárias.
- b) Determine os atributos de cada classe.
- c) Defina em qual classe deve ficar a responsabilidade de verificar a disponibilidade de um livro.
- d) Defina em qual classe deve ficar a responsabilidade de verificar o limite de empréstimos do usuário.

- e) Explique por que a data prevista para devolução não deve ser um atributo da classe `Livro`.
- f) Represente os relacionamentos entre as classes.

Depois, implemente a modelagem e crie um programa principal que realize empréstimos, devoluções e consultas.

29. Uma plataforma de entrega de refeições funciona da seguinte maneira:

Cada restaurante possui nome, endereço e um cardápio. O cardápio é formado por produtos, e cada produto possui código, descrição e preço.

Um cliente possui nome, telefone e endereço de entrega. Para realizar uma compra, o cliente cria um pedido e adiciona produtos do cardápio. Para cada produto adicionado, o pedido deve armazenar também a quantidade solicitada.

O pedido deve calcular seu valor total, permitir a remoção de itens e possuir uma situação, que pode ser “Aberto”, “Confirmado”, “Em preparação”, “Saiu para entrega” ou “Entregue”. Um pedido confirmado não pode receber novos itens.

Faça a modelagem orientada a objetos do problema.

Sua resposta deve identificar:

- as classes;
- os atributos de cada classe;
- os métodos de cada classe;
- quais relacionamentos representam associação;
- quais relacionamentos representam composição;
- qual classe deve calcular o subtotal de um produto com determinada quantidade;
- qual classe deve calcular o valor total da compra.

Depois, implemente as classes e crie um programa principal que:

- crie um restaurante e seu cardápio;
- crie dois clientes;
- registre pedidos com diferentes itens;
- altere as situações dos pedidos;
- tente adicionar um item a um pedido já confirmado;
- exiba um resumo completo de cada pedido.

30. Analise os requisitos de um estacionamento:

O estacionamento possui várias vagas numeradas. Cada vaga possui um tipo, que pode ser comum, para motocicleta ou para pessoa com deficiência, e pode estar livre ou ocupada.

Cada veículo possui placa, modelo e tipo. Quando um veículo entra, deve ser direcionado a uma vaga compatível. O sistema registra o horário de entrada. Na saída, registra o horário de saída, calcula o tempo de permanência e determina o valor a pagar.

O estacionamento deve informar a quantidade de vagas livres por tipo, localizar o veículo por sua placa, impedir que dois veículos ocupem a mesma vaga e impedir que o mesmo veículo seja registrado duas vezes.

Antes da implementação:

- a) Identifique as possíveis classes do sistema.
- b) Diferencie as informações que pertencem ao veículo, à vaga e ao registro de permanência.
- c) Defina os métodos de cada classe.
- d) Indique qual classe deve localizar uma vaga disponível.
- e) Indique qual classe deve calcular o valor da permanência.
- f) Explique por que o horário de entrada não deve ser um atributo permanente da classe `Veículo`.

Implemente a solução e crie um programa principal que simule entradas e saídas, incluindo tentativas inválidas.

- 31.** Uma universidade deseja informatizar parte de seu processo de matrícula. Considere a descrição:

Cada aluno possui matrícula, nome e curso. Cada disciplina possui código, nome e carga horária. A oferta de uma disciplina em determinado semestre é representada por uma turma. A turma possui número, semestre, professor, limite de vagas e uma lista de alunos matriculados.

Um aluno pode se matricular em várias turmas, e uma turma pode possuir vários alunos. Entretanto, o aluno não pode se matricular duas vezes na mesma turma. Uma matrícula somente pode ser realizada quando houver vagas disponíveis.

A turma deve permitir a matrícula e o cancelamento da matrícula de um aluno. O sistema também deve consultar as turmas de um aluno, informar a quantidade de vagas restantes de uma turma e gerar uma lista de chamada.

Realize a modelagem orientada a objetos do sistema.

Inicialmente, responda:

- a) Quais substantivos do texto representam possíveis classes?
- b) Quais substantivos representam apenas atributos de outras classes?
- c) Quais ações do texto representam possíveis métodos?
- d) Qual é a diferença entre uma *Disciplina* e uma *Turma*?
- e) Em qual classe deve ficar o método de matricular um aluno?
- f) Como deve ser representada a relação entre alunos e turmas?
- g) Qual objeto deve verificar se ainda existem vagas?

Em seguida, defina as classes com seus atributos, construtores e métodos.

No programa principal:

- crie diferentes disciplinas;
- crie pelo menos duas turmas;
- crie diferentes alunos;
- realize matrículas e cancelamentos;
- tente repetir uma matrícula;
- tente matricular um aluno em uma turma cheia;
- exiba a lista de chamada de cada turma;
- exiba as turmas em que cada aluno está matriculado.

Parte 2: Questões de Múltipla Escolha (Padrão ENADE)

32. (**Assertão e Razão**) Na Engenharia de Software, a escolha entre algoritmos recursivos e iterativos impacta diretamente o consumo de recursos da máquina, sendo uma decisão arquitetural importante em sistemas embarcados.

Avalie as asserções a seguir e a relação proposta entre elas:

- I. Em sistemas com restrição severa de memória, desenvolvedores devem dar preferência ao uso de estruturas de repetição (*loops* baseados em `while` ou `for`) para processamento contínuo, evitando o uso de algoritmos baseados em recursividade profunda.

PORQUE

- II. Funções recursivas geram empilhamento de chamadas (*Call Stack*), onde o interpretador aloca novos espaços de memória de contexto a cada evocação sucessiva até encontrar o caso base, podendo ocasionar um erro crítico de *Stack Overflow*.

A respeito dessas asserções, assinale a opção correta:

- A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- E) As asserções I e II são proposições falsas.

33. **(Resposta Múltipla)** O Tratamento de Exceções é um pilar da construção de sistemas resilientes, garantindo que eventos anômalos não interrompam a execução do software de forma catastrófica. Considere a prática de capturar exceções utilizando blocos genéricos, como no código: `except Exception: pass`.

Com base nas boas práticas de desenvolvimento de software em Python, avalie as afirmações a seguir:

- I. O uso do bloco `except Exception: pass` é recomendado na camada final de interfaces gráficas para garantir que, independentemente do erro ocorrido no banco de dados, o usuário final não visualize telas de travamento.
- II. O tratamento genérico de exceções mascara erros críticos do sistema (*bug masking*), impedindo que o desenvolvedor identifique falhas estruturais, como variáveis nulas ou erros de tipagem, dificultando o processo de depuração (*debugging*).
- III. A abordagem arquitetural correta exige a captura de classes de exceção específicas (ex: `FileNotFoundException`, `ValueError`), permitindo que o sistema tome ações de contingência direcionadas para cada cenário de falha.

É correto o que se afirma em:

- A) I, apenas.
- B) III, apenas.
- C) I e II, apenas.
- D) II e III, apenas.
- E) I, II e III.

34. **(Resposta Múltipla)** A manipulação de arquivos de entrada e saída (I/O) exige gerenciamento seguro de recursos pelo Sistema Operacional. Em Python, a instrução `with open("dados.txt", "w") as arquivo:` é o padrão da indústria. Avalie as razões técnicas que justificam a adoção obrigatória deste formato (Gerenciador de Contexto):

- I. O bloco `with` assegura que o arquivo será implicitamente fechado (`close()`) ao fim do escopo, eliminando o risco de corrompimento de dados na memória primária.
- II. Em caso de uma interrupção inesperada (Exceção) durante a escrita dos dados dentro do bloco, o `with` suprime o fechamento do arquivo para permitir que a equipe técnica acesse o modo de recuperação do Sistema Operacional.
- III. O modo `"w"` (*write*) garante a preservação do histórico, anexando as novas inserções textuais ao final do arquivo sem sobrescrever os logs preexistentes.

É correto o que se afirma em:

- A) I, apenas.
 - B) II, apenas.
 - C) I e III, apenas.
 - D) II e III, apenas.
 - E) I, II e III.
35. **(Interpretação)** Observe a arquitetura de controle de fluxo de exceções implementada a seguir:

```
1 def fechar_balanco(faturamento_str):  
2     try:  
3         faturamento = float(faturamento_str)  
4         imposto = faturamento * 0.1  
5     except ValueError:  
6         print("Erro de tipagem detectado.")  
7     else:  
8         print("Balanco consolidado com sucesso.")  
9     finally:  
10        print("Fechando modulo de calculo.")
```

Caso a função seja acionada passando o argumento `fechar_balanco("1500.50")`, qual será a sequência exata de saídas impressas no console?

- A) "Erro de tipagem detectado."seguido de "Fechando modulo de calculo."
- B) "Balanco consolidado com sucesso."
- C) Apenas "Fechando modulo de calculo."
- D) "Balanco consolidado com sucesso."seguido de "Fechando modulo de calculo."
- E) Nenhuma mensagem é exibida, pois a conversão `float` não lança exceção.

36. **(Aserção e Razão)** O paradigma da Orientação a Objetos (POO) utiliza a modelagem do mundo real para construir abstrações de software por meio de Classes e Objetos, onde o conceito de encapsulamento protege as informações sensíveis da aplicação.

Avalie as asserções a seguir e a relação proposta entre elas:

- I. Modificar diretamente os atributos da instância em escopo global (ex: `cliente.saldo = 5000`) viola princípios de engenharia de software e segurança.

PORQUE

- II. Alterações de estado interno devem ocorrer exclusivamente por meio de métodos comportamentais definidos pela Classe (ex: `cliente.depositar(5000)`), os quais contêm as devidas abstrações de regras de negócio, auditoria e levantamento de exceções.

A respeito dessas asserções, assinale a opção correta:

- A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- E) As asserções I e II são proposições falsas.
37. **(Aserção e Razão)** A modelagem recursiva demanda a identificação da condição de parada antes do desenvolvimento da lógica iterativa, para garantir a viabilidade matemática e computacional do algoritmo.

Avalie as asserções a seguir e a relação proposta entre elas:

- I. Uma função recursiva projetada para buscar um usuário em um banco de dados sempre retornará um valor válido se a rotina de autoinvocação (chamada a si mesma) estiver correta no código, dispensando o Caso Base.

PORQUE

- II. O Caso Base em algoritmos recursivos atua apenas como uma estrutura de tratamento de erros (exceções lógicas), sendo acionado unicamente quando o interpretador da linguagem falha ao executar os *statements* principais da função.

A respeito dessas asserções, assinale a opção correta:

- A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.

- B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- E) As asserções I e II são proposições falsas.